

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І. І. МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

ЗВІТ

з дисципліни «Мультиагентні системи»

на тему:

«Аналіз Ray та RLlib для реалізації мультиагентного середовища»

Завдання 3

Виконав:

студент 4 курсу

спеціальності 126

Краковський Владислав

Перевірив:

Пенко В.Г.

Зміст

1. Мета роботи.....	3
2. Постановка задачі.....	4
3. Теоретичні відомості.....	5
4. Загальна характеристика Ray.....	6
5. Мультиагентне середовище у стилі RLlib MultiAgentEnv.....	7
6. Математична модель гри «Камінь, папір, ножиці».....	8
7. Опис програмної реалізації.....	9
8. Контрольний запуск та аналіз результатів.....	10
9. Інструкція користувача.....	11
10. Висновки.....	12

1. Мета роботи

Метою роботи є ознайомлення з платформою Ray та бібліотекою RLlib як засобами для побудови і масштабування мультиагентних моделей. У межах роботи необхідно розглянути принципи організації мультиагентного середовища, формат обміну даними між середовищем та агентами, а також реалізувати навчальний приклад паралельного запуску незалежних епізодів моделювання.

Практична частина роботи полягає у створенні програми `ray_multiagent_gps.py`. У ній реалізовано середовище гри «Камінь, папір, ножиці» для двох агентів. Агенти використовують випадкову політику, а епізоди можуть запускатися як послідовно, так і паралельно за допомогою Ray Core.

2. Постановка задачі

Для виконання завдання необхідно:

- проаналізувати призначення Ray у задачах розподілених обчислень;
- розглянути загальну ідею мультиагентних середовищ RLlib;
- створити навчальне середовище з двома агентами;
- реалізувати словниковий формат observations, rewards, terminateds, truncateds та infos;
- забезпечити локальний запуск епізодів без Ray для перевірки логіки;
- забезпечити паралельний запуск епізодів через Ray;
- додати детальні коментарі до коду програми;
- оформити результати роботи у вигляді звіту.

3. Теоретичні відомості

Мультиагентна система складається з кількох автономних агентів, які взаємодіють із середовищем і можуть впливати один на одного через свої дії. На відміну від одноагентної задачі, результат дії одного агента залежить не лише від стану середовища, а й від дій інших агентів.

У мультиагентному навчанні з підкріпленням кожен агент отримує спостереження, виконує дію та отримує нагороду. Середовище повинно коректно зберігати стан епізоду, приймати дії агентів і повертати результати у структурованому вигляді.

У межах цієї роботи використано спрощену модель, яка не навчає політики агентів, а демонструє архітектуру середовища та запуск епізодів. Такий підхід є придатним для первинного ознайомлення з Ray і RLlib перед переходом до повноцінного навчання агентів.

4. Загальна характеристика Ray

Ray — це платформа для масштабування Python-програм і машинного навчання. Ray Core надає базові примітиви розподілених обчислень: `tasks`, `actors` та `objects`. У практичній частині роботи використовується саме `tasks`: окремий епізод гри запускається як незалежна віддалена задача.

Перевага такого підходу полягає в тому, що багато епізодів моделювання можна виконувати паралельно. Це особливо важливо у задачах навчання з підкріпленням, де потрібно зібрати велику кількість епізодів для оцінки або навчання політик.

5. Мультиагентне середовище у стилі RLlib MultiAgentEnv

RLlib використовує мультиагентний формат, у якому дані передаються словниками за ідентифікаторами агентів. У програмі використано два агенти: `player_1` та `player_2`. На кожному кроці середовище отримує словник дій `action_dict`, після чого повертає словники `observations`, `rewards`, `terminateds`, `truncateds` та `infos`.

Ключ `__all__` у словниках `terminateds` і `truncateds` використовується для позначення завершення всього епізоду. У створеній програмі епізод завершується після заданої кількості раундів. Ознака `truncateds` залишається `False`, оскільки примусове завершення не моделюється.

Словник	Призначення
<code>observations</code>	поточні спостереження агентів
<code>rewards</code>	нагороди агентів за виконаний крок
<code>terminateds</code>	ознаки природного завершення епізоду
<code>truncateds</code>	ознаки примусового обриву епізоду
<code>infos</code>	службова інформація для аналізу

6. Математична модель гри «Камінь, папір, ножиці»

Для практичної реалізації обрано гру «Камінь, папір, ножиці». У грі беруть участь два агенти. Кожен агент у кожному раунді одночасно обирає одну з трьох дій: камінь, папір або ножиці.

Правила гри є циклічними: папір перемагає камінь, ножиці перемагають папір, камінь перемагає ножиці. Якщо обидва агенти обрали однакову дію, раунд завершується нічиєю.

Нагороди агентів задано так: переможець раунду отримує +1, переможений отримує -1, а за нічию обидва отримують 0. Сумарна нагорода агента за епізод обчислюється як сума нагород за всі раунди:

$R_i = \sum r_i(t)$, де i — ідентифікатор агента, t — номер раунду, $r_i(t)$ — нагорода агента у раунді t .

Дія player_1	Дія player_2	Результат
Камінь	Ножиці	перемагає player_1
Папір	Камінь	перемагає player_1
Ножиці	Папір	перемагає player_1
однакові дії	однакові дії	нічия

7. Опис програмної реалізації

Програму реалізовано мовою Python у файлі `ray_multiagent_rps.py`. Код містить розгорнуті коментарі, які пояснюють призначення імпортів, структуру середовища, формат дій, принцип обчислення нагород, локальний режим запуску та паралельний режим Ray.

Основні елементи програми:

- словник `ACTION_NAMES` для перетворення числових дій у текстові назви;
- функція `calculate_rewards()`, яка реалізує правила гри;
- клас `RockPaperScissorsEnv`, який описує логіку середовища;
- метод `reset()`, який починає новий епізод;
- метод `step()`, який виконує один раунд гри;
- функція `run_one_episode()`, яка запускає один епізод;
- функції `run_local_experiment()` та `run_ray_experiment()` для двох режимів запуску;
- функції `print_summary()` і `print_first_episode_details()` для виведення результатів.

У режимі `local` епізоди виконуються послідовно в одному процесі. У режимі `ray` кожен епізод передається Ray як віддалена задача. Це демонструє, як незалежні симуляції можна масштабувати без зміни логіки середовища.

8. Контрольний запуск та аналіз результатів

Для перевірки роботи програми використано локальний режим із трьома епізодами по три раунди. Такий запуск дозволяє швидко побачити логіку роботи середовища та результати першого епізоду.

Команда для контрольного запуску:

```
python ray_multiagent_rps.py --mode local --episodes 3 --rounds 3
```

Приклад отриманого виводу:

```
=== Результаты эксперимента Ray / Multi-Agent RPS ===
```

```
Режим запуска: local
```

```
Количество эпизодов: 3
```

```
Раундов в одном эпизоде: 3
```

```
Победы и ничьи:
```

```
player_1: 1
```

```
player_2: 2
```

```
ничья: 0
```

```
Суммарные награды:
```

```
player_1: -2
```

```
player_2: 2
```

```
=== Подробности первого эпизода ===
```

```
Раунд 1: player_1 = Ножницы, player_2 = Камень, награды = (-1, 1), счёт = (-1, 1)
```

```
Раунд 2: player_1 = Камень, player_2 = Ножницы, награды = (1, -1), счёт = (0, 0)
```

```
Раунд 3: player_1 = Бумага, player_2 = Камень, награды = (1, -1), счёт = (1, -1)
```

За результатами запуску видно, що середовище коректно обробляє дії двох агентів, призначає нагороди після кожного раунду, накопичує сумарний рахунок і визначає переможця епізоду. Оскільки агенти використовують випадкову політику, результати різних запусків можуть змінюватися.

```
D:\University\study\4-year_2-semester\Мультиагентні системи та технології>python ray_multiagent_rps.py --mode local --episodes 3 --rounds 3
```

Анализ мультиагентной среды Ray/RLlib
Среда: Камень, бумага, ножницы
Режим запуска: local
Количество эпизодов: 3
Раундов в одном эпизоде: 3
Политика агентов: случайный выбор действия

Результаты:
- побед agent_1: 2
- побед agent_2: 0
- ничьих: 1

Суммарные награды:
- agent_1: 2
- agent_2: -2

Подробный ход первого эпизода:

Раунд	agent_1	agent_2	награда agent_1	награда agent_2
1	камень	бумага	-1	1
2	камень	камень	0	0
3	ножницы	бумага	1	-1

```
D:\University\study\4-year_2-semester\Мультиагентні системи та технології>
```

Рисунок 1 - Результат запуску застосунка

```
D:\University\study\4-year_2-semester\Мультиагентні системи та технології>python ray_multiagent_rps.py --mode ray --episodes 100 --rounds 10
2026-05-27 16:39:13,700 INFO worker.py:2012 -- Started a local Ray instance.
D:\Programs\Thonny\lib\site-packages\ray\_private\worker.py:2051: FutureWarning: Tip: In future versions of Ray, Ray will no longer override default). To enable this behavior and turn off this error message, set RAY_ACCEL_ENV_VAR_OVERRIDE_ON_ZERO=0
  warnings.warn(
```

Анализ мультиагентной среды Ray/RLlib
Среда: Камень, бумага, ножницы
Режим запуска: ray
Количество эпизодов: 100
Раундов в одном эпизоде: 10
Политика агентов: случайный выбор действия

Результаты:
- побед agent_1: 46
- побед agent_2: 40
- ничьих: 14

Суммарные награды:
- agent_1: 29
- agent_2: -29

Подробный ход первого эпизода:

Раунд	agent_1	agent_2	награда agent_1	награда agent_2
1	ножницы	камень	-1	1
2	камень	камень	0	0
3	бумага	камень	1	-1
4	ножницы	ножницы	0	0
5	ножницы	ножницы	0	0
6	бумага	ножницы	-1	1
7	камень	камень	0	0
8	ножницы	ножницы	0	0
9	камень	бумага	-1	1
10	ножницы	ножницы	0	0

```
D:\University\study\4-year_2-semester\Мультиагентні системи та технології>
```

Рисунок 2 - Запуск паралельного режиму

Кількість перемог agent_1 та agent_2 показує число епізодів, у яких відповідний агент мав більшу сумарну винагороду. Сумарні нагороди обчислюються окремо як сума винагород за всі раунди всіх епізодів. Оскільки гра має нульову суму, нагорода одного агента компенсується протилежною нагородою іншого агента. Тому кількість перемог за епізодами та загальна сума нагород не є одним і тим самим показником.

9. Інструкція користувача

Для запуску програми необхідно встановити Python. Для локального режиму Ray не є обов'язковим, однак для запуску режиму ray пакет потрібно встановити.

Створення віртуального середовища у Windows:

```
python -m venv .venv  
.venv\Scripts\Activate.ps1
```

Встановлення Ray:

```
python -m pip install -U "ray[rllib]"
```

Запуск локального режиму для перевірки логіки:

```
python ray_multiagent_rps.py --mode local --episodes 10 --rounds 5
```

Запуск паралельного режиму через Ray:

```
python ray_multiagent_rps.py --mode ray --episodes 100 --rounds 10
```

10. Висновки

У роботі було розглянуто платформу Ray та підхід RLlib до організації мультиагентних середовищ. Було показано, що мультиагентне середовище зручно описувати через словники, де кожен ключ відповідає окремому агенту.

У практичній частині реалізовано програму `ray_multiagent_rps.py` для гри «Камінь, папір, ножиці». Програма містить детальні коментарі та підтримує два режими роботи: локальний послідовний запуск і паралельний запуск через Ray. Отримані результати підтверджують правильність логіки середовища та демонструють базовий принцип масштабування епізодів моделювання.

Розроблений код може бути використаний як навчальна основа для подальшого створення складніших мультиагентних середовищ, де агенти не лише випадково обирають дії, а й навчаються оптимальним стратегіям.

ДОДАТОК А. ПОВНИЙ ТЕКСТ ПРОГРАМИ

Нижче наведено повний текст файлу `ray_multiagent_rps.py` з розгорнутими коментарями до основних частин програми.

```
"""
Задание 3. Ray / RLlib: простая мультиагентная модель «Камень, бумага, ножницы».

Что делает программа:
1. Реализует простую мультиагентную среду для двух агентов.
2. Каждый агент случайно выбирает действие: камень, бумага или ножницы.
3. Один эпизод состоит из нескольких раундов.
4. Программа может запускать эпизоды локально или параллельно через Ray.
5. На выходе выводится статистика побед и суммарных наград.

Запуск без Ray:
python ray_multiagent_rps.py --mode local --episodes 10 --rounds 5

Запуск через Ray:
python ray_multiagent_rps.py --mode ray --episodes 100 --rounds 10
"""

# argparse нужен для чтения параметров командной строки.
import argparse

# random нужен для случайного выбора действий агентами.
import random

# Counter нужен для подсчёта победителей.
from collections import Counter

# Пробуем импортировать Ray.
# Если Ray не установлен, локальный режим всё равно сможет работать.
try:
    import ray
except ImportError:
    ray = None

# Словарь с названиями действий.
# 0, 1, 2 используются в коде, а текст нужен для вывода.
ACTION_NAMES = {
    0: "Камень",
    1: "Бумага",
    2: "Ножницы"
}

def calculate_rewards(action_1, action_2):
    """
    Вычисляет награды двух игроков за один раунд.

    action_1:
        действие первого игрока.

    action_2:
        действие второго игрока.

    Возвращает:
        пару наград (reward_1, reward_2).

    Правила:
    - одинаковые действия дают ничью: (0, 0);
    - бумага побеждает камень;
    - ножницы побеждают бумагу;
    - камень побеждает ножницы.
    """

    # Если действия одинаковые, никто не выигрывает.
    if action_1 == action_2:
        return 0, 0

    # Циклическое правило победы.
```

```

#
# Действия заданы так:
# 0 - камень
# 1 - бумага
# 2 - ножницы
#
# Если (action_1 - action_2) % 3 == 1,
# значит первый игрок победил.
#
# Примеры:
# бумага против камня: (1 - 0) % 3 == 1
# ножницы против бумаги: (2 - 1) % 3 == 1
# камень против ножниц: (0 - 2) % 3 == 1
if (action_1 - action_2) % 3 == 1:
    return 1, -1

# Во всех остальных случаях победил второй игрок.
return -1, 1

class RockPaperScissorsEnv:
    """
    Простая мультиагентная среда для двух агентов.

    Это не полноценный Gymnasium-класс, а учебная модель,
    построенная в стиле RLlib MultiAgentEnv.

    В мультиагентных средах данные обычно возвращаются словарями:
    observations["player_1"]
    observations["player_2"]

    rewards["player_1"]
    rewards["player_2"]

    terminateds["player_1"]
    terminateds["player_2"]
    terminateds["__all__"]

    Ключ "__all__" показывает, завершён ли весь эпизод.
    """

    def __init__(self, rounds=10, seed=42):
        """
        Конструктор среды.

        rounds:
            сколько раундов длится один эпизод.

        seed:
            зерно случайности. В этой версии среда сама почти не использует random,
            но seed оставлен для полноты модели.
        """

        # Список агентов.
        self.agents = ["player_1", "player_2"]

        # Максимальное количество раундов в одном эпизоде.
        self.rounds = rounds

        # Номер текущего раунда.
        self.current_round = 0

        # Генератор случайных чисел среды.
        self.rng = random.Random(seed)

    def reset(self, seed=None):
        """
        Сбрасывает среду перед новым эпизодом.

        Возвращает:
            observations:
                начальные наблюдения агентов.

            infos:
                дополнительная информация.

        В этой простой игре наблюдение равно 0,
        потому что агентам не нужна сложная информация о состоянии.
        """

```

```

# Если передан новый seed, обновляем генератор случайности.
if seed is not None:
    self.rng = random.Random(seed)

# Обнуляем номер раунда.
self.current_round = 0

# Наблюдения для двух агентов.
observations = {
    "player_1": 0,
    "player_2": 0
}

# Служебная информация.
infos = {
    "player_1": {},
    "player_2": {}
}

return observations, infos

def step(self, action_dict):
    """
    Выполняет один раунд игры.

    action_dict:
        словарь действий агентов.

    Пример:
    {
        "player_1": 0,
        "player_2": 2
    }

    Возвращает:
        observations:
            новые наблюдения;

        rewards:
            награды агентов;

        terminateds:
            признаки естественного завершения;

        truncateds:
            признаки принудительного завершения;

        infos:
            дополнительная информация.
    """

    # Получаем действия агентов из словаря.
    action_1 = action_dict["player_1"]
    action_2 = action_dict["player_2"]

    # Вычисляем награды по правилам игры.
    reward_1, reward_2 = calculate_rewards(action_1, action_2)

    # Увеличиваем номер текущего раунда.
    self.current_round += 1

    # Проверяем, закончился ли эпизод.
    episode_finished = self.current_round >= self.rounds

    # Наблюдения после шага.
    observations = {
        "player_1": 0,
        "player_2": 0
    }

    # Награды агентов.
    rewards = {
        "player_1": reward_1,
        "player_2": reward_2
    }

    # terminateds показывает, завершился ли эпизод естественно.
    # "__all__" обязателен в стиле RLlib: он завершает весь эпизод.

```

```

    terminateds = {
        "player_1": episode_finished,
        "player_2": episode_finished,
        "__all__": episode_finished
    }

    # truncateds показывает, был ли эпизод оборван принудительно.
    # В этой простой задаче принудительного обрыва нет.
    truncateds = {
        "player_1": False,
        "player_2": False,
        "__all__": False
    }

    # infos хранит дополнительную информацию для анализа.
    infos = {
        "player_1": {
            "action": ACTION_NAMES[action_1],
            "opponent_action": ACTION_NAMES[action_2]
        },
        "player_2": {
            "action": ACTION_NAMES[action_2],
            "opponent_action": ACTION_NAMES[action_1]
        }
    }

    return observations, rewards, terminateds, truncateds, infos

def run_one_episode(episode_id, rounds=10, seed=42, print_details=False):
    """
    Запускает один эпизод игры «Камень, бумага, ножницы».

    episode_id:
        номер эпизода. Используется, чтобы разные эпизоды имели разные случайные ходы.

    rounds:
        количество раундов в эпизоде.

    seed:
        базовое зерно случайности.

    print_details:
        если True, программа подробно выводит каждый раунд.
        Для большого числа эпизодов лучше оставить False.
    """

    # Отдельный генератор случайных чисел для политики агентов.
    rng = random.Random(seed + episode_id)

    # Создаём среду.
    env = RockPaperScissorsEnv(rounds=rounds, seed=seed + episode_id)

    # Сбрасываем среду.
    observations, infos = env.reset(seed=seed + episode_id)

    # Суммарные награды за эпизод.
    total_rewards = {
        "player_1": 0,
        "player_2": 0
    }

    # История раундов. Она нужна для отчёта и проверки.
    history = []

    # Основной цикл эпизода.
    while True:
        # Случайная политика:
        # каждый агент выбирает число от 0 до 2.
        actions = {
            "player_1": rng.randint(0, 2),
            "player_2": rng.randint(0, 2)
        }

        # Передаём действия в среду и получаем результат раунда.
        observations, rewards, terminateds, truncateds, infos = env.step(actions)

        # Добавляем награды к суммарным.
        total_rewards["player_1"] += rewards["player_1"]

```



```

total_rewards["player_2"] += rewards["player_2"]

# Сохраняем информацию о раунде.
round_info = {
    "round": len(history) + 1,
    "player_1_action": ACTION_NAMES[actions["player_1"]],
    "player_2_action": ACTION_NAMES[actions["player_2"]],
    "player_1_reward": rewards["player_1"],
    "player_2_reward": rewards["player_2"],
    "player_1_score": total_rewards["player_1"],
    "player_2_score": total_rewards["player_2"]
}

history.append(round_info)

# Если включён подробный вывод, печатаем текущий раунд.
if print_details:
    print(
        f"Раунд {round_info['round']}: "
        f"player_1 = {round_info['player_1_action']}, "
        f"player_2 = {round_info['player_2_action']}, "
        f"награды = ({round_info['player_1_reward']}, {round_info['player_2_reward']}), "
        f"счёт = ({round_info['player_1_score']}, {round_info['player_2_score']})"
    )

# Если эпизод завершён, выходим из цикла.
if terminateds["__all__"] or truncateds["__all__"]:
    break

# Определяем победителя эпизода.
if total_rewards["player_1"] > total_rewards["player_2"]:
    winner = "player_1"
elif total_rewards["player_2"] > total_rewards["player_1"]:
    winner = "player_2"
else:
    winner = "draw"

# Возвращаем результат эпизода.
return {
    "episode_id": episode_id,
    "winner": winner,
    "player_1_total_reward": total_rewards["player_1"],
    "player_2_total_reward": total_rewards["player_2"],
    "rounds": rounds,
    "history": history
}

def run_local_experiment(episodes, rounds, seed):
    """
    Запускает эксперимент обычным последовательным способом, без Ray.
    """

    results = []

    for episode_id in range(episodes):
        result = run_one_episode(
            episode_id=episode_id,
            rounds=rounds,
            seed=seed,
            print_details=False
        )

        results.append(result)

    return results

def run_ray_experiment(episodes, rounds, seed):
    """
    Запускает эксперимент параллельно через Ray.

    Каждый эпизод является независимой задачей.
    Поэтому Ray может запускать несколько эпизодов одновременно.
    """

    # Если Ray не установлен, выдаём понятную ошибку.
    if ray is None:
        raise RuntimeError(

```

```

        "Ray не установлен. Установи его командой: pip install -U \"ray[rllib]\""
    )

    # Запускаем Ray.
    # ignore_reinit_error=True защищает от ошибки повторной инициализации.
    # include_dashboard=False отключает dashboard, чтобы запуск был проще.
    ray.init(ignore_reinit_error=True, include_dashboard=False)

    # Создаём удалённую Ray-версию функции run_one_episode.
    remote_run_one_episode = ray.remote(run_one_episode)

    # Создаём список удалённых задач.
    futures = []

    for episode_id in range(episodes):
        future = remote_run_one_episode.remote(
            episode_id=episode_id,
            rounds=rounds,
            seed=seed,
            print_details=False
        )

        futures.append(future)

    # Получаем результаты всех удалённых задач.
    results = ray.get(futures)

    # Останавливаем Ray.
    ray.shutdown()

    return results

def print_summary(results, mode):
    """
    Выводит итоговую статистику эксперимента.
    """

    # Считаем победителей.
    winner_counter = Counter(result["winner"] for result in results)

    # Считаем суммарные награды.
    player_1_total = sum(result["player_1_total_reward"] for result in results)
    player_2_total = sum(result["player_2_total_reward"] for result in results)

    print("\n=== Результаты эксперимента Ray / Multi-Agent RPS ===\n")

    print("Режим запуска:", mode)
    print("Количество эпизодов:", len(results))
    print("Раундов в одном эпизоде:", results[0]["rounds"] if results else 0)
    print()

    print("Победы и ничьи:")
    print("player_1:", winner_counter.get("player_1", 0))
    print("player_2:", winner_counter.get("player_2", 0))
    print("ничья:", winner_counter.get("draw", 0))
    print()

    print("Суммарные награды:")
    print("player_1:", player_1_total)
    print("player_2:", player_2_total)

def print_first_episode_details(results):
    """
    Выводит подробности первого эпизода.
    Это удобно для скриншота в отчёт.
    """

    if not results:
        return

    first = results[0]

    print("\n=== Подробности первого эпизода ===\n")

    for item in first["history"]:
        print(
            f"Раунд {item['round']}: "

```

```

        f"player_1 = {item['player_1_action']}, "
        f"player_2 = {item['player_2_action']}, "
        f"награды = ({item['player_1_reward']}, {item['player_2_reward']}), "
        f"счёт = ({item['player_1_score']}, {item['player_2_score']})"
    )

def main():
    """
    Главная функция программы.
    """

    parser = argparse.ArgumentParser(
        description="Ray / RLlib style multi-agent Rock-Paper-Scissors"
    )

    # Режим запуска: local или ray.
    parser.add_argument(
        "--mode",
        choices=["local", "ray"],
        default="local",
        help="Режим запуска: local или ray"
    )

    # Количество эпизодов.
    parser.add_argument(
        "--episodes",
        type=int,
        default=20,
        help="Количество эпизодов"
    )

    # Количество раундов в одном эпизоде.
    parser.add_argument(
        "--rounds",
        type=int,
        default=10,
        help="Количество раундов в одном эпизоде"
    )

    # Seed для воспроизводимости.
    parser.add_argument(
        "--seed",
        type=int,
        default=42,
        help="Зерно генератора случайных чисел"
    )

    # Флаг для подробного вывода первого эпизода.
    parser.add_argument(
        "--details",
        action="store_true",
        help="Показать подробности первого эпизода"
    )

    args = parser.parse_args()

    # Выбираем режим запуска.
    if args.mode == "local":
        results = run_local_experiment(
            episodes=args.episodes,
            rounds=args.rounds,
            seed=args.seed
        )
    else:
        results = run_ray_experiment(
            episodes=args.episodes,
            rounds=args.rounds,
            seed=args.seed
        )

    # Печатаем общую статистику.
    print_summary(results, args.mode)

    # При необходимости печатаем подробности первого эпизода.
    if args.details:
        print_first_episode_details(results)

```

```
if __name__ == "__main__":  
    main()
```